Design Rationale: Mini Library Management System (Python)

This document explains the design choices made for the Mini Library Management System developed for the Object-Oriented Programming 1 course. The system was built using Python and employs core data structures such as dictionaries, lists, and tuples to efficiently manage books, members, and library operations like adding, searching, updating, deleting, borrowing, and returning books.

1. Design Approach
The system follows a procedural structure with well-defined functions that handle each major operation (CRUD and borrow/return). The design ensures modularity, where each function performs a single, clear task. This approach improves readability, debugging, and reusability of code while maintaining the integrity of stored data.

2. Data Structures Used

a. Dictionary: The *dictionary* was chosen to store book details because it allows fast access, updates, and deletion using unique ISBN keys. Each book record contains fields such as title, author, genre, and total copies, making it easy to organize and retrieve data.

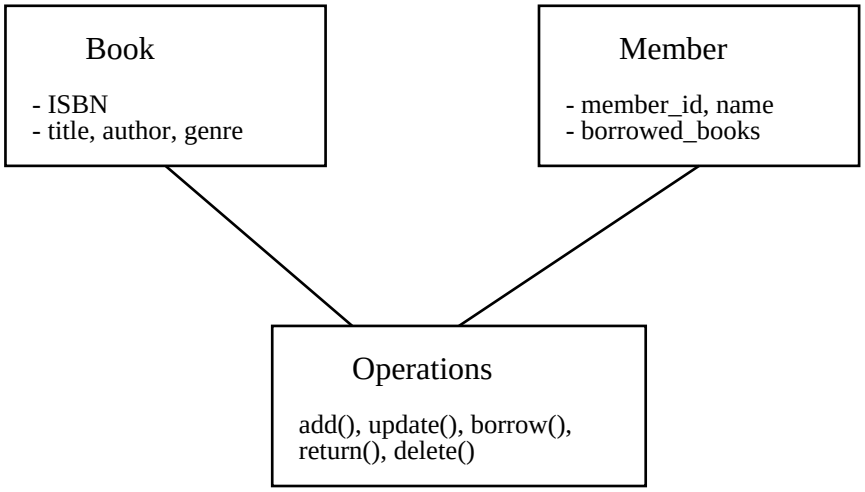
b. List of Dictionaries: A *list* was used to store member information. Each member record is represented as a dictionary containing the member ID, name, email, and borrowed books. The list structure allows flexible addition and iteration through members.

c. Tuple: The *tuple* was used to store valid book genres (e.g., Fiction, Non-Fiction, Sci-Fi). Since genres are fixed and should not change during program execution, a tuple ensures data immutability and integrity.

3. Functional Design
Each core operation was implemented as a standalone function to maintain simplicity and clarity. Functions like *add_book()*, *search_books()*, *borrow_book()*, and *return_book()* directly interact with the data structures, ensuring data consistency and preventing invalid actions (e.g., borrowing unavailable books or exceeding borrowing limits).

4. UML Design Summary
The UML diagram represents the logical relationship between key components: the Book and Member data structures, and the main operations that interact with them. It visually shows how each function manipulates or retrieves information from the system.

5. Conclusion
The combination of dictionaries, lists, and tuples provides a balanced approach to achieving flexibility, speed, and reliability in managing library operations. The system is modular, easy to extend, and well-suited for beginner-level Object-Oriented Programming projects. These design decisions align with the assignment's learning objectives of demonstrating understanding of Python data structures and functional programming principles.


Book
- ISBN
- title, author, genre
Member
- member_id, name
- borrowed_books
Operations
add(), update(), borrow(),
return(), delete()